

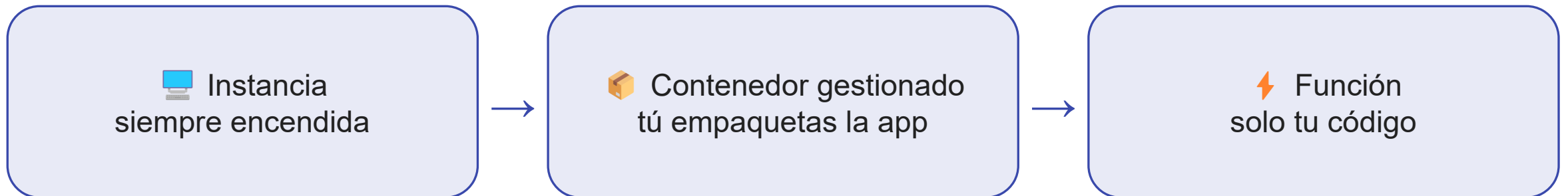
Serverless

El peldaño más alto de la escalera

Todo lo desplegado hasta ahora tiene, de fondo, una instancia encendida esperando peticiones, aunque no llegue ninguna. En la 5.3 lo viste en la factura: dos t3.small cuestan lo mismo con mil visitas que con ninguna.

Hoy: código que solo existe mientras se ejecuta

Lo dispara un evento concreto y no hay ningún servidor que administres.



El contenedor gestionado lo verás en la próxima sesión — hoy, el otro extremo.

Modelo de eventos y ejecución efímera

Una función Lambda

Un trozo de código que AWS ejecuta por ti cuando ocurre algo — un evento. En cuanto termina, su entorno se apaga hasta el siguiente.



Subida de una foto
a S3



Función Lambda
se crea, ejecuta, desaparece



Miniatura
generada

Una instancia EC2 es como tener la cocina de un restaurante encendida todo el día, con el cocinero dentro, haya o no comensales. Una función Lambda es un cocinero que aparece cuando llega un pedido, cocina ese plato y se va: no pagas por el tiempo que la cocina está vacía.

Qué puede disparar una función

Quién dispara la función	Ejemplo en Escaparate
Un objeto nuevo en S3	Se sube la foto de un producto y se genera su miniatura
Una petición HTTP (pasarela de API)	Una URL que devuelve los datos de una foto en JSON
Un reloj (regla programada)	Cada noche, borrar los carritos abandonados de hace más de una semana
Un mensaje en una cola	Cada pedido nuevo envía un correo de confirmación

Hoy trabajas con el primer caso: un objeto nuevo en S3.

Anatomía de una función Lambda

Pieza	Qué es	En la Actividad 6.2
Entorno de ejecución (runtime)	El lenguaje: Python, Node.js, Java...	Python
Controlador (handler)	La función por la que Lambda empieza a ejecutar tu código	lambda_handler
Evento	Los datos del aviso, en JSON	Qué objeto se ha subido y a qué bucket
Rol de ejecución	Los permisos con los que corre la función	LabRole
Paquete de código	Tu código más las librerías, en un .zip	Tu código y Pillow

Antes del código: la forma del evento

Lo que S3 le manda a Lambda (event), simplificado

```
{
  "Records": [
    {
      "s3": {
        "bucket": {"name": "..."},
        "object": {"key": "..."}
      }
    }
  ]
}
```

Records es una lista

S3 puede avisar de varios objetos en un solo evento; por eso el código la recorre con un for.

Niveles anidados

bucket y key están varios niveles abajo: se baja por el diccionario, nivel a nivel, hasta llegar al dato.

Con esta forma ya identificada, el código que la lee tiene sentido en la siguiente diapositiva.

El esqueleto mínimo de una función

```
def lambda_handler(event, context):  
    # S3 puede avisar de varios objetos en un solo evento  
    for registro in event["Records"]:  
        bucket = registro["s3"]["bucket"]["name"]  
        clave = registro["s3"]["object"]["key"]  
        print(f"Objeto nuevo: {bucket}/{clave}")  
    return {"statusCode": 200}
```

Un esqueleto: no genera nada todavía

Solo lee el evento y escribe un log. La lógica real (generar la miniatura con Pillow) la ves completa en la Actividad 6.2.

print no se ve en pantalla: va a CloudWatch Logs, el mismo servicio de la 5.1. Es el primer sitio donde mirar cuando una función no hace lo esperado.

context (segundo parámetro, obligatorio): metadatos internos de la invocación que aquí no hace falta usar.

¿Y ese return, si nadie llama por HTTP?

`{"statusCode": 200}` solo lo necesita quien espera una respuesta, como el cliente de una invocación síncrona a través de la pasarela.

Disparada por S3, nadie lee lo que devuelve

Lambda solo necesita que la función termine sin lanzar ninguna excepción para darla por bien ejecutada.

Si lanzara una excepción, sí importaría

Es lo que dispara el reintento de la invocación asíncrona, que ves en la siguiente diapositiva.

Por qué Pillow se empaqueta en CloudShell

Ya incluida

Lambda trae por defecto boto3, la librería de AWS para Python.

Hay que empaquetarla

Pillow (procesa imágenes) no viene incluida, y tiene partes compiladas para un sistema operativo concreto.

Por eso CloudShell, y no tu ordenador

Pillow debe empaquetarse en un Linux compatible con Lambda. CloudShell ya es Amazon Linux; tu Windows o macOS no lo son.

Es exactamente lo que haces en el Paso 3 de la Actividad 6.2.

La función que se dispara a sí misma

El bucle infinito que factura cada vuelta

Si una función se activa cuando aparece un objeto nuevo en un bucket y escribe su resultado en ese mismo bucket, cada miniatura que genera es otro objeto nuevo que la vuelve a disparar — y así sin parar.

Se evita filtrando por prefijo

escaparate/

Dispara la función: fotos originales

miniaturas/

Genera la función: no la vuelve a disparar

metadatos/

Genera la función: no la vuelve a disparar

Es el filtro que configuras en el Paso 3 de la Actividad 6.2.

Invocación síncrona frente a asíncrona

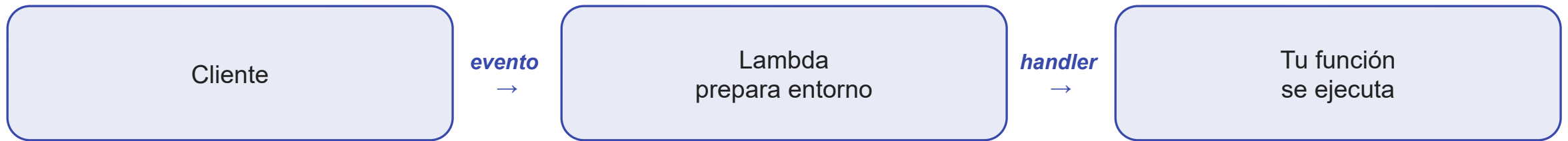
	Invocación síncrona	Invocación asíncrona
Ejemplo	Una petición HTTP por la pasarela	La subida de una foto a S3
Quién espera la respuesta	El cliente, con la conexión abierta	Nadie: S3 solo avisa y sigue
Si la función falla	El error vuelve al cliente	Lambda reintenta hasta dos veces más

Por eso una función que se dispara desde S3 debe poder ejecutarse dos veces sobre la misma foto sin causar daño: generar la miniatura otra vez es inofensivo, cobrar un pedido dos veces no lo es.

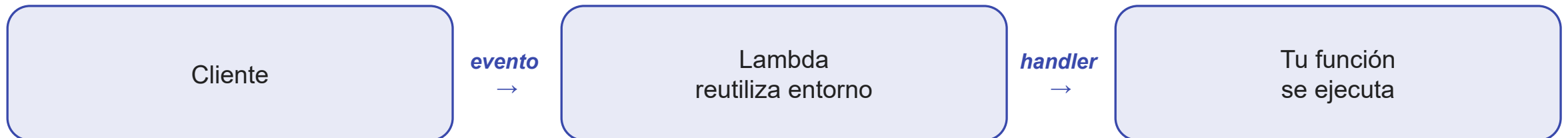
Arranque en frío: la primera invocación

La primera vez (o tras un rato sin uso), Lambda tiene que preparar un entorno y cargar tu código antes de ejecutarlo. Las siguientes invocaciones, si llegan enseguida, reutilizan el entorno ya preparado: van «en caliente».

Primera petición (en frío)



Segunda petición, después (en caliente)



0,84 s

primera petición (fría), medido en el Lab

0,06 s

las siguientes (calientes)

Tres límites que acotan qué tareas caben

Concepto	Valor	Consecuencia
Tiempo máximo de ejecución	15 min por invocación (3 s por defecto)	No sirve para tareas largas, como convertir un vídeo de una hora
Memoria	De 128 MB a 10.240 MB	Tú eliges la cantidad; la CPU disponible crece con ella
Tamaño del paquete	50 MB el .zip, 250 MB descomprimido	Una librería enorme puede no caber

Más memoria acelera, pero no siempre abarata

La memoria también reparte CPU. Si una función tarda la mitad al doblar la memoria, cuesta lo mismo y responde el doble de rápido. Pero solo pasa así cuando la función pasa casi todo el tiempo calculando.

Memoria	Duración (caliente)	Coste por invocación
128 MB	≈ 120 ms	≈ 0,015 GB-segundo
512 MB	≈ 90 ms	≈ 0,044 GB-segundo

La función de miniaturas de la 6.2 pasa buena parte del tiempo esperando a S3

Más rápida con más memoria, pero también más cara por invocación. El ajuste de memoria es probar con datos, no adivinar — lo harás tú mismo en el Paso 2.

Facturación por invocación

No existe «encendida»

Se cobra por petición y por GB-segundo: la memoria asignada (en GB) multiplicada por los segundos que dura la ejecución.

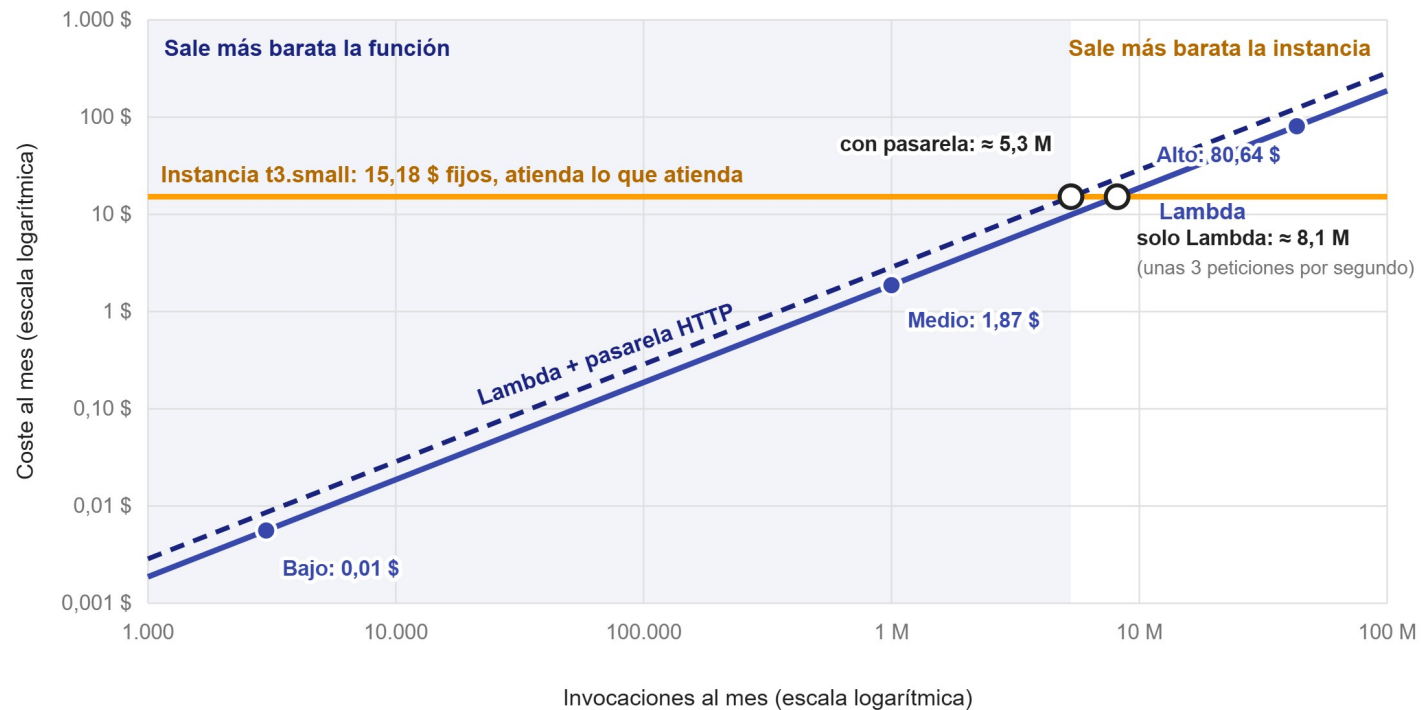
Concepto	Precio
Peticiones	0,20 \$ por cada millón
Cómputo	0,0000166667 \$ por GB-segundo
Capa gratuita mensual	1 millón de peticiones y 400.000 GB-segundo

Precios de US East (N. Virginia), arquitectura x86.

El cruce con una instancia t3.small

Coste mensual: función Lambda frente a una instancia

Función de 512 MB y 200 ms de media, sin capa gratuita · instancia t3.small · US East (N. Virginia)



Cero invocaciones, cero coste de cómputo — pero no cero coste

El bucket de S3 sigue facturando con su propia unidad. Y una función que se dispara sin control sí que llena la factura.

Pasarela de API: la puerta de entrada HTTP

Para que una función responda a una petición HTTP hace falta algo delante que la reciba y la convierta en un evento: cumple un papel parecido al del balanceador de carga para tus instancias.



	API HTTP	API REST
Precio	1,00 \$ por millón	3,50 \$ por millón
Qué ofrece	Lo básico: rutas, integración	Lo básico, más tres funciones extra
Cuándo	La opción por defecto — la usarás en la 6.2	Cuando necesitas alguna función extra

Las tres funciones extra de API REST

Claves de acceso

Cada aplicación lleva su propia clave: puedes dar (o cortar) una cuota de peticiones a cada cliente por separado.

Caché de respuestas

Si la misma petición llega dos veces seguidas, la pasarela devuelve la respuesta guardada, sin invocar la función de nuevo.

Transformación

Cambia el formato de lo que entra o sale (por ejemplo, adapta el JSON para un cliente antiguo) sin tocar el código.

Para la mini API de la Actividad 6.2 no necesitas ninguna de las tres, así que usarás API HTTP.

El peldaño más extremo de la escalera



Instancia (gestionas el sistema operativo entero) → Contenedor gestionado (próxima sesión) → Función (solo tu código). En cada peldaño delegas más operación a cambio de menos control.

Cuándo compensa serverless, y cuándo no

Situación	¿Encaja?
Tarea puntual disparada por un evento	Sí, el caso ideal
Tráfico irregular: picos y horas sin peticiones	Sí, no pagas los huecos
Carga constante y alta, muchas invocaciones por segundo	A menudo no: una instancia sale más barata
Proceso de más de 15 minutos	No, choca con el límite de ejecución
Guarda datos en el propio servidor entre peticiones	No, cada ejecución empieza de cero

En la Actividad 6.2 mides la latencia y el coste reales de la misma operación con una función y con una instancia, y decides tú, con esos datos.

Ideas clave

1

Una función serverless no existe hasta que un evento la dispara, y desaparece al terminar: sin servidor y sin coste de cómputo mientras nadie la invoca.

2

Una función Lambda tiene un controlador, un evento en JSON, un rol de ejecución y un paquete con código y librerías.

3

Una función que escribe en el bucket que la dispara puede llamarse a sí misma sin parar: se evita filtrando por prefijo.

4

El arranque en frío añade latencia a la primera invocación; tiempo, memoria y tamaño del paquete acotan qué tareas caben.

5

Se factura por invocaciones y GB-segundo: con tráfico bajo o irregular sale más barato que una instancia; con carga alta y constante, puede salir más caro.

6

Una pasarela de API convierte peticiones HTTP en invocaciones, con un papel parecido al del balanceador de carga.

7

Instancia, contenedor gestionado y función son tres peldaños de la misma escalera: el criterio es qué necesita la carga.